# CodeMentor AI — Nasıl Çalışır?

Statik Analiz + Yerel Derleyici + Çift Geçişli LLM Mimarisi (K0 - K6 Katman Raporu)

7 KATMANLI MIMARI | DETERMINISTIK TEMEL | %0 ÇÖZÜM SIZINTISI | 33 ADR STANDARDI

## 1. Mimari Akış Şeması




## 2. Katman Katman Çalışma Prensibi

### Katman 0: Giriş & Normalizasyon (K0)

Kullanıcı kodu sisteme girdiğinde satır sonları (CRLF/LF) ve karakter kodlaması (UTF-8) standartlaştırılır. Dosya uzantısından veya içerik imzalarından programlama dili ve oyun motoru bağlamı (Unity / Unreal) otomatik saptanır. 50 KB üzerindeki girdiler güvenlik gereği reddedilir.

### Katman 1: Ön Doğrulama & Erken Çıkış (K1)

**Tier 0 (tree-sitter):** Kodun AST (Soyut Sözdizimi Ağacı) parse edilir. `ERROR` veya `MISSING` düğümleri varsa sözdizimi hatası Türkçe anlaşılır tanılarla döner. **LLM hiç çağrılmaz (~1 ms erken çıkış).**

**Tier 1 (Yerel Derleyiciler):** Sistemde gcc, clang veya dotnet/Roslyn kuruluysa, kod çalıştırılmadan yalnızca sözdizimi/tip denetimi (`-fsyntax-only`) yapılır.

### Katman 2: Deterministik Statik Analiz (K2)

tree-sitter sorgularıyla koddan yapısal olgular toplanır: döngü derinliği, çağrı grafiği, dinamik bellek tahsisleri ve 41 kurallık katalog taranır (C-MEM-001 sızıntı, C-NUM-001 sıfıra bölme vb.). Deterministik Big-O zaman ve alan karmaşıklığı hesaplanır.

### Katman 3: Güvenli Prompt Tasarımı (K3)

Kullanıcı kodu **asla güvenilmeyen veri** kabul edilir. Prompt injection girişimlerini önlemek için kod `<user_code>` etiketleri içinde izole edilir. K2'den elde edilen statik olgular prompt'a eklenerek modelin halüsinasyon görmesi engellenir (Big-O doğruluğu %85'ten %96'ya yükselir).

### Katman 4 & 5: Çift Geçişli LLM ve Sızıntı Koruması (K4 - K5)

- **Geçiş A (Derin Analiz):** LLM önce kodun mantıksal hatalarını, sınır durumlarını ve karmaşıklığını gerekçeleriyle analiz eder. Çıktı JSON şeması ve satır kanıtı testleriyle doğrulanır.
- **Geçiş B (Sokratik Mentorluk):** Analiz bulguları pedagojik sorulara dönüştürülür.
- **Sızıntı Koruması (Leakage Guard):** Üretilen sorular regex ve kural filtrelerinden geçer. Kodu doğrudan düzelten veya cevabı açık eden soru derhal reddedilir ve yeniden ürettirilir (%0 sızıntı).

### Katman 6: Çok Formatlı Raporlama (K6)

Tüm analiz tek bir birleşik veri modelinde toplanır. Kullanıcı ihtiyacına göre JSON, Markdown, endüstri standardı SARIF 2.1.0 veya zengin React Web / Windows Masaüstü arayüzlerinde görüntülenir.

## 3. Kademeli İpucu Sistemi (Progressive Hinting)

| İpucu Seviyesi | Kapsam | Örnek Mentor Yaklaşımı |
|---|---|---|
| **Seviye 1: Kavramsal** | Hatanın kavramsal mantığı | "Fonksiyon heap'ten bellek ayırdığında bu kaynağın yaşam döngüsü ne zaman biter?" |

| | | |
|---|---|---|
| **Seviye 2: Satır Bağlamı** | İlgili satır ve değişkenler | "8. satırdaki döngü bittiğinde `tmp` işaretçisinin gösterdiği alan serbest bırakıldı mı?" |
| **Seviye 3: Yönlendirme** | Çözüm yönü (kod verilmez) | "Fonksiyon return satırına ulaşmadan önce tahsis edilen alanı serbest bırakacak standart C fonksiyonunu hatırla." |